



École Supérieure de l'Éducation et de la Formation
Oujda — ESEF



RAPPORT DE PROJET

M&C Fashion

Développement d'un Site E-Commerce

Réalisées par :

Maroua Foulko
Chaymae Jellouli

Encadrée par :

Pr. Sarra Roubi

Technologies Utilisées

PHP

MySQL

HTML5

CSS3

JavaScript

Année Universitaire : 2025 / 2026

Description du Projet

Ce projet consiste à développer un site e-commerce moderne permettant la gestion des produits, des utilisateurs, du panier, des favoris et des commandes en ligne. Le projet applique également le responsive design, des interactions dynamiques et la sécurité des données avec PHP et MySQL.

Table des matières

1	Base de données	4
1.1	Nom de la base de données	4
1.2	Table les utilisateurs (users)	4
1.3	Table produits (products)	5
1.4	Table Commandes (orders)	5
1.5	Table ligne_Commandes (order_items)	6
1.6	Table Favoris (wishlist)	6
1.7	Table Messages_Contact (contact_messages)	7
2	Fichier index.php	8
2.1	Initialisation et actions	8
2.2	Gestion du panier (Add to Cart)	9
2.3	Gestion de la Wishlist	10
2.4	Hero Slider HTML	10
2.5	Affichage des produits	11
2.6	Section Best Sellers	12
2.7	Section Call To Action (CTA)	13
3	Fichier header.php	14
3.1	Comptage du panier	14
3.2	Comptage de la wishlist	14
3.3	Structure HTML principale	15
3.4	Navigation et actions du header	16
3.5	Menu utilisateur connecté	16
3.6	Menu hamburger mobile	17
4	Fichier footer.php	18
4.1	Colonne logo et réseaux sociaux	18
4.2	Liens rapides	19
4.3	Mon compte	19
4.4	Informations de contact	20
4.5	Copyright dynamique	20
4.6	Scripts JavaScript	20
5	Fichier db.php	22

6	Fichier auth.php	24
6.1	Initialisation et vérification de session	24
6.2	Déconnexion	24
6.3	Connexion utilisateur	25
6.4	Inscription utilisateur	26
6.5	Formulaires HTML	26
6.6	Changement d'onglet JavaScript	27
7	Fichier cart.php	28
7.1	Initialisation du panier	28
7.2	Augmenter la quantité	28
7.3	Diminuer la quantité	29
7.4	Supprimer un article	29
7.5	Vider le panier	30
7.6	Calcul du total	30
7.7	Affichage des articles	31
8	Fichier wishlist.php	32
8.1	Initialisation	32
8.2	Suppression d'un favori	32
8.3	Ajout au panier depuis la wishlist	33
8.4	Récupération des produits favoris	34
8.5	Synchronisation BDD/Session	34
8.6	Affichage des produits favoris	35
9	Fichier product.php	36
9.1	Récupération et validation du produit	36
9.2	Produits similaires et état wishlist	37
9.3	Affichage des détails produit	37
9.4	Boutons Add to Cart et Wishlist	38
9.5	Produits similaires	38
10	Fichier checkout.php	40
10.1	Protection de la page	40
10.2	Calcul du total	41
10.3	Traitement de la commande	41
10.4	Affichage de la confirmation	42
10.5	Formulaire checkout	43
11	Fichier orders.php	44
11.1	Protection et récupération des commandes	44
11.2	Affichage de la liste des commandes	45
11.3	Modal détails commande	46
12	Fichier contact.php	47
12.1	Traitement du formulaire	47
12.2	Informations de contact et formulaire	48

13 Fichier style.css	50
13.1 Variables globales CSS	50
13.2 Reset et styles de base	50
13.3 Boutons	51
13.4 Header	51
13.5 Hero Slider	52
13.6 Grille produits et interactions	52
13.7 Autres styles	53
14 Fichier main.js	55
14.1 Scroll et menu mobile	55
14.2 Slider automatique	56
14.3 Filtres produits	57
14.4 Système de modals	57
14.5 Messages flash et onglets auth	58

Chapitre 1

Base de données

1.1 Nom de la base de données

Nom de la base de données : `m&c_fashion`

Cette base de données permet de gérer :

- les produits
- les utilisateurs
- les commandes
- les favoris
- les messages de contact

1.2 Table les utilisateurs (users)

La table `users` permet de stocker les informations des utilisateurs du site. Elle est utilisée pour :

- l'inscription
- la connexion
- la gestion des comptes utilisateurs

Listing 1.1 – Table users

```
CREATE TABLE users (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  username    VARCHAR(100) NOT NULL UNIQUE,  
  email       VARCHAR(150) NOT NULL UNIQUE,  
  password    VARCHAR(255) NOT NULL,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

1.3 Table produits (products)

La table **products** contient tous les produits affichés dans la boutique en ligne. Elle permet :

- l’affichage dynamique des produits
- la gestion du stock
- l’affichage des images
- l’organisation des catégories

Listing 1.2 – Table products

```
CREATE TABLE products (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(255)      NOT NULL,  
  description  TEXT              NOT NULL,  
  price       DECIMAL(10,2)     NOT NULL,  
  image       VARCHAR(255)      NOT NULL,  
  category    VARCHAR(100)      NOT NULL,  
  stock       INT DEFAULT 0,  
  is_bestseller TINYINT(1) DEFAULT 0,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

1.4 Table Commandes (orders)

La table **orders** permet de stocker les commandes des utilisateurs. Elle contient :

- les informations du client
- l’adresse
- le montant total
- l’état de la commande

Listing 1.3 – Table orders

```
CREATE TABLE orders (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  user_id     INT                NOT NULL,  
  customer_name VARCHAR(255)     NOT NULL,  
  customer_email VARCHAR(255)    NOT NULL,  
  customer_address TEXT          NOT NULL,  
  total_amount DECIMAL(10,2)     NOT NULL,  
  status      VARCHAR(50) DEFAULT 'pending',  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
  ON DELETE CASCADE  
);
```

1.5 Table ligne_Commandes (order_items)

La table `order_items` contient les détails des produits présents dans chaque commande. Elle permet :

- gérer plusieurs produits dans une commande
- enregistrer les quantités
- stocker les prix des produits commandés

Listing 1.4 – Table `order_items`

```
CREATE TABLE order_items (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  order_id    INT          NOT NULL,  
  product_id  INT          NOT NULL,  
  product_name VARCHAR(255) NOT NULL,  
  product_price DECIMAL(10,2) NOT NULL,  
  quantity    INT          NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,  
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE  
);
```

1.6 Table Favoris (wishlist)

La table `wishlist` permet aux utilisateurs d'ajouter des produits dans leur liste de favoris. Elle améliore l'expérience utilisateur en permettant :

- sauvegarder produits préférés
- retrouver produits plus tard

Listing 1.5 – Table `wishlist`

```
CREATE TABLE wishlist (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  user_id     INT NOT NULL,  
  product_id  INT NOT NULL,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE  
);
```

1.7 Table Messages_Contact (contact_messages)

La table `contact_messages` stocke les messages envoyés depuis le formulaire de contact du site. Elle permet :

- communication avec visiteurs
- réception des questions clients
- gestion du support utilisateur

Listing 1.6 – Table `contact_messages`

```
CREATE TABLE contact_messages (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL,  
  email       VARCHAR(255) NOT NULL,  
  message     TEXT          NOT NULL,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Chapitre 2

Fichier index.php

Le fichier `index.php` représente la page principale du site web M&C Fashion. C'est le centre principal du projet car il permet d'afficher les produits et les différentes sections de la boutique.

Il assure :

- l'affichage dynamique des produits
- la navigation principale du site
- l'interaction avec le panier et la wishlist
- la communication avec la base de données

Le fichier fonctionne avec plusieurs autres fichiers comme : `db.php`, `header.php`, `footer.php`, `product.php`, `cart.php`.

2.1 Initialisation et actions

Le fichier commence par inclure la base de données, démarrer la session et initialiser le panier et la wishlist. Il traite ensuite les actions GET avant d'afficher la page.

Listing 2.1 – Initialisation et récupération des données

```
include 'db.php';
session_start();

if (!isset($_SESSION['cart']))
    $_SESSION['cart'] = [];
if (!isset($_SESSION['wishlist']))
    $_SESSION['wishlist'] = [];

$products = $pdo->query(
    "SELECT * FROM products ORDER BY id ASC"
)->fetchAll();
$bestsellers = $pdo->query(
    "SELECT * FROM products WHERE is_bestseller = 1 LIMIT 3"
)->fetchAll();
```

```

if (isset($_SESSION['user_id'])) {
    $wRows = $pdo->prepare(
        "SELECT product_id FROM wishlist WHERE user_id = ?"
    );
    $wRows->execute([$_SESSION['user_id']]);
    $_SESSION['wishlist'] =
        array_column($wRows->fetchAll(), 'product_id');
}

```

2.2 Gestion du panier (Add to Cart)

Lorsqu'un utilisateur clique sur le bouton **"Add to Cart"**, le système récupère le produit sélectionné depuis la base de données puis l'ajoute au panier utilisateur grâce aux sessions PHP. Si le produit existe déjà dans le panier, la quantité augmente automatiquement. Après l'ajout, un message de confirmation apparaît et le panier est mis à jour dynamiquement.

Listing 2.2 – Action : Ajouter au panier

```

if (isset($_GET['action']) &&
    $_GET['action'] === 'add' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    $stmt = $pdo->prepare(
        "SELECT * FROM products WHERE id = ?"
    );
    $stmt->execute([$id]);
    $product = $stmt->fetch();
    if ($product) {
        $found = false;
        foreach ($_SESSION['cart'] as &$item) {
            if ($item['id'] == $id) {
                $item['qty']++;
                $found = true;
                break;
            }
        }
        unset($item);
        if (!$found) {
            $product['qty'] = 1;
            $_SESSION['cart'][] = $product;
        }
        $_SESSION['flash'] = [
            'type' => 'success',
            'msg' => "" . $product['name'] . " added to cart!"
        ];
    }
    header("Location: index.php");
    exit;
}

```

2.3 Gestion de la Wishlist

Cette partie du code permet de gérer la **Wishlist** du site. Lorsqu'un utilisateur clique sur l'icône cœur, le produit est ajouté ou supprimé des favoris automatiquement. Pour les utilisateurs connectés, la wishlist est enregistrée dans MySQL. Pour les invités, elle est stockée dans la session.

Listing 2.3 – Action : Toggle Wishlist

```
if (isset($_GET['action']) &&
    $_GET['action'] === 'toggle_wishlist' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    if (isset($_SESSION['user_id'])) {
        $check = $pdo->prepare(
            "SELECT id FROM wishlist
            WHERE user_id=? AND product_id=?"
        );
        $check->execute([$_SESSION['user_id'], $id]);
        if ($check->rowCount() > 0) {
            $pdo->prepare(
                "DELETE FROM wishlist
                WHERE user_id=? AND product_id=?"
            )->execute([$_SESSION['user_id'], $id]);
        } else {
            $pdo->prepare(
                "INSERT INTO wishlist (user_id, product_id)
                VALUES (?, ?)"
            )->execute([$_SESSION['user_id'], $id]);
        }
    } else {
        if (in_array($id, $_SESSION['wishlist'])) {
            $_SESSION['wishlist'] =
                array_diff($_SESSION['wishlist'], [$id]);
        } else {
            $_SESSION['wishlist'][] = $id;
        }
    }
    header("Location: index.php");
    exit;
}
```

2.4 Hero Slider HTML

Le Hero est la section principale affichée en haut de la page. Il contient trois slides avec des textes différents, des boutons de navigation (précédent/suivant) et des points de navigation (dots). Le slider est animé automatiquement grâce à `main.js`.

Listing 2.4 – Section Hero Slider

```

<section class="hero">
  <div class="hero-slide_active"
    style="background-image:url('assests/mcfashionimage/
    SLIDER.png');">
    <div class="hero-content">
      <span class="hero-tag">Up to 75% off</span>
      <h1>Discover &<br>Special Offer<br>Fashion</h1>
      <a href="#products" class="btn">Shop Now</a>
    </div>
  </div>

  <div class="hero-slide"
    style="background-image:url('assests/mcfashionimage/
    SLIDER.png');">
    <div class="hero-content">
      <span class="hero-tag">New Collection</span>
      <h1>Style &<br>Comfort<br>Everyday</h1>
      <a href="#products" class="btn">Explore</a>
    </div>
  </div>

  <button class="slider-prev" id="sliderPrev">&#8592;</button>
  <button class="slider-next" id="sliderNext">&#8594;</button>
  <div class="slider-dots">
    <button class="slider-dot_active"></button>
    <button class="slider-dot"></button>
    <button class="slider-dot"></button>
  </div>
</section>

```

2.5 Affichage des produits

Le système récupère les produits depuis la base de données et les affiche sous forme de cartes dynamiques. Chaque carte contient : l'image, le nom, le prix, la catégorie et des boutons overlay (panier, wishlist, détail). Les boutons de filtre permettent de trier par catégorie sans recharger la page.

Listing 2.5 – Grille des produits avec filtres

```

<div class="filters">
  <button class="filter-btn_active"
    data-filter="all">All</button>
  <button class="filter-btn"
    data-filter="men">Men</button>
  <button class="filter-btn"
    data-filter="women">Women</button>
</div>

<div class="products-grid">
  <?php foreach ($products as $p):
    $is_wished = in_array($p['id'], $_SESSION['wishlist']);

```

```

?>
<div class="product-card"
  data-cat="<?=htmlspecialchars($p['category'])_?>"
  <div class="product-img-wrap">
    "
    loading="lazy">
    <span class="cat-badge">
      <?= ucfirst($p['category']) ?>
    </span>
    <div class="product-overlay">
      <a href="index.php?action=add&id=<?= $p['id']_?>"
        class="overlay-btn">
        <i class="fas fa-shopping-cart"></i>
      </a>
      <a href="index.php?action=toggle_wishlist&id=
      <?= $p['id']_?>"
        class="overlay-btn
      <?= $is_wished_?_wished'_:_'_?>"
        <i class="fas fa-heart"></i>
      </a>
      <a href="product.php?id=<?= $p['id']_?>"
        class="overlay-btn">
        <i class="fas fa-eye"></i>
      </a>
    </div>
  </div>
  <div class="product-info">
    <h4><?= htmlspecialchars($p['name']) ?></h4>
    <span class="product-price">
      <?= number_format($p['price'], 2) ?> MAD
    </span>
    <a href="index.php?action=add&id=<?= $p['id']_?>"
      class="btn btn-outline btn-sm">Add to Cart</a>
  </div>
</div>
<?php endforeach; ?>
</div>

```

2.6 Section Best Sellers

Cette partie du code permet d'afficher la section **Best Sellers** dans la page d'accueil du site. Le système récupère automatiquement les produits marqués `is_bestseller = 1` dans la base de données (limité à 3) puis les affiche sous forme de cartes produits avec un bouton **“View Details”**.

Listing 2.6 – Section Best Sellers

```

<?php if (!empty($bestsellers)): ?>
<section class="section_section-alt" id="bestsellers">
  <div class="container">
    <div class="section-header">

```

```

    <h2>Best Sellers</h2>
    <div class="section-divider"></div>
  </div>
  <div class="products-grid">
    <?php foreach ($bestsellers as $p): ?>
      <div class="product-card">
        <div class="product-img-wrap">
          "
          <span class="cat-badge">
            <?=$_ucfirst($p['category'])_?>
          </span>
        </div>
        <div class="product-info">
          <h4><?=$_htmlspecialchars($p['name'])_?></h4>
          <span class="product-price">
            <?=$_number_format($p['price'], 2)_?> MAD
          </span>
          <a href="product.php?id=<?=$_$p['id']_?>"
            class="btn btn-sm">View Details</a>
        </div>
      </div>
    <?php endforeach; ?>
  </div>
</section>
<?php endif; ?>

```

2.7 Section Call To Action (CTA)

Cette partie du code représente une section *Call To Action* (CTA) dans la page d'accueil du site. Elle encourage les visiteurs à rejoindre la communauté du site en créant un compte utilisateur.

Le bouton "Create Free Account" redirige l'utilisateur vers la page d'inscription `auth.php`. Le code inclut également le fichier `footer.php` afin d'afficher le pied de page commun dans toutes les pages du site web.

Listing 2.7 – Section CTA et inclusion footer

```

<section class="section" style="background: #2d5a27;
color: #fff; text-align: center; padding: 70px 20px;">
  <h2>Join Our Community</h2>
  <p>Get exclusive access to new arrivals.</p>
  <a href="auth.php?tab=register" class="btn btn-gold">
    Create Free Account
  </a>
</section>

<?php include 'includes/footer.php'; ?>

```

Chapitre 3

Fichier header.php

Le fichier `header.php` représente l'en-tête principal du site web. Il est affiché dans toutes les pages du projet grâce à `include()`.

Ce fichier permet de gérer :

- la barre de navigation
- le logo du site
- le panier
- la wishlist
- la connexion utilisateur
- le menu responsive mobile

3.1 Comptage du panier

Listing 3.1 – Comptage articles panier

```
$cart_count = 0;
if (isset($_SESSION['cart'])) {
    foreach ($_SESSION['cart'] as &$item) {
        $cart_count += $item['qty'];
    }
}
```

Lorsqu'un utilisateur ajoute des produits au panier, le système parcourt automatiquement tous les produits enregistrés dans la session panier afin de calculer le nombre total des articles. Le résultat est ensuite affiché dans le badge du panier présent dans le header du site web.

3.2 Comptage de la wishlist

Listing 3.2 – Comptage articles wishlist

```

$wish_count = 0;
if (isset($_SESSION['user_id']) && isset($pdo)) {
    $wStmt = $pdo->prepare(
        "SELECT COUNT(*) FROM wishlist WHERE user_id=?"
    );
    $wStmt->execute([$_SESSION['user_id']]);
    $wish_count = $wStmt->fetchColumn();
} elseif (isset($_SESSION['wishlist'])) {
    $wish_count = count($_SESSION['wishlist']);
}

```

Les produits ajoutés dans la wishlist sont comptés automatiquement afin d'afficher le nombre total des favoris dans le header du site. Lorsque l'utilisateur est connecté, les données sont récupérées depuis la base de données. Sinon, les favoris sont sauvegardés temporairement dans les sessions PHP pour conserver l'expérience utilisateur.

3.3 Structure HTML principale

Listing 3.3 – En-tête HTML du site

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport"
        content="width=device-width, initial-scale=1.0" />
    <title>
        <?= isset($page_title)
            ? $page_title . ' - M&C Fashion'
            : 'M&C Fashion' ?>
    </title>
    <link href="https://fonts.googleapis.com/css2?..."
        rel="stylesheet" />
    <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/
font-awesome/6.1.2/css/all.min.css" />
    <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/
swiper@8/swiper-bundle.min.css" />
    <link rel="stylesheet" href="assets/style.css" />
    <?= isset($extra_css) ? $extra_css : '' ?>
</head>

```

Cette partie du code représente l'en-tête HTML principal du site web. Elle permet de configurer la structure de la page, le titre du site et le responsive design pour les différents appareils.

Le code importe également :

- les polices Google Fonts pour améliorer le design
- les icônes Font Awesome utilisées dans le site

— le fichier CSS principal pour le style et l'apparence du site web

3.4 Navigation et actions du header

Listing 3.4 – Navigation principale et boutons header

```
<header class="site-header" id="site-header">
  <div class="container_header-inner">
    <a href="index.php" class="logo">
      M<em>&amp;C</em> <span>FASHION</span>
    </a>
    <nav class="main-nav" id="main-nav">
      <a href="index.php"
        class="<?=$_$current_page=='index.php'?'active':''$_?>"
        Home
      </a>
      <a href="index.php#products">Products</a>
      <a href="index.php#bestsellers">Best Sellers</a>
      <a href="contact.php"
        class="<?=$_$current_page=='contact.php'?'active':''$_?>"
        Contact
      </a>
    </nav>
    <div class="header-actions">
      <a href="wishlist.php" class="icon-btn" title="My_Wishlist">
        <i class="fas_fa-heart"></i>
        <?php if ($wish_count > 0): ?>
          <span class="badge"><?=$_$wish_count ?></span>
        <?php endif; ?>
      </a>
      <a href="cart.php" class="icon-btn" title="My_Cart">
        <i class="fas_fa-shopping-bag"></i>
        <?php if ($cart_count > 0): ?>
          <span class="badge"><?=$_$cart_count ?></span>
        <?php endif; ?>
      </a>
    </div>
  </div>
</header>
```

Cette partie du code représente les principales actions du header du site web. Elle permet d'afficher : le logo du site, la barre de navigation, la wishlist et le panier. Le système affiche également les badges dynamiques pour montrer le nombre de produits ajoutés dans les favoris et le panier afin d'améliorer l'expérience utilisateur et la navigation dans le site.

3.5 Menu utilisateur connecté

Listing 3.5 – Menu utilisateur connecté / bouton login

```
<?php if (isset($_SESSION['user'])): ?>
  <div class="user-menu">
    <button class="icon-btn user-toggle" title="Account">
      <i class="fas fa-user-circle"></i>
    </button>
    <div class="user-dropdown">
      <span class="user-name">
        Hello, <?= htmlspecialchars($_SESSION['user']) ?>
      </span>
      <a href="orders.php">
        <i class="fas fa-box"></i> My Orders
      </a>
      <a href="wishlist.php">
        <i class="fas fa-heart"></i> Wishlist
      </a>
      <a href="auth.php?action=logout" class="logout-link">
        <i class="fas fa-sign-out-alt"></i> Logout
      </a>
    </div>
  </div>
<?php else: ?>
  <a href="auth.php" class="btn-login">Login</a>
<?php endif; ?>
```

Cette partie du code permet de vérifier si un utilisateur est connecté au site web grâce aux sessions PHP. Si l'utilisateur est connecté, le système affiche automatiquement le menu du compte contenant : le nom utilisateur, les commandes, la wishlist et le bouton logout. Sinon, le bouton **Login** apparaît afin de permettre l'accès à la page d'authentification.

3.6 Menu hamburger mobile

Listing 3.6 – Bouton hamburger responsive

```
<button class="menu-toggle" id="menu-toggle"
  aria-label="Open menu">
  <span></span><span></span><span></span>
</button>
</div></div></header>
<div class="header-spacer"></div>
```

Cette partie du code crée le bouton hamburger utilisé pour la navigation responsive du site web sur les appareils mobiles. Lorsque l'utilisateur clique sur ce bouton, le menu navigation s'affiche automatiquement afin de faciliter la navigation sur les petits écrans. Le **header-spacer** permet également de corriger l'espace sous le header fixe afin d'éviter que le contenu du site soit caché derrière la barre de navigation.

Chapitre 4

Fichier footer.php

Le fichier `footer.php` représente le pied de page du site web. Il est affiché dans toutes les pages du projet afin de regrouper :

- les informations du site
- les liens rapides
- les réseaux sociaux
- les informations de contact
- les scripts JavaScript

4.1 Colonne logo et réseaux sociaux

Listing 4.1 – Logo et réseaux sociaux dans le footer

```
<footer class="site-footer">
  <div class="container_footer-grid">
    <div class="footer-col">
      <h3 class="logo-footer">
        M<em>&amp;C <span>FASHION</span>
      </h3>
      <p>Your destination for timeless and contemporary
        fashion. Style for every moment.</p>
      <div class="social-links">
        <a href="#" aria-label="Instagram">
          <i class="fab fa-instagram"></i>
        </a>
        <a href="#" aria-label="Facebook">
          <i class="fab fa-facebook-f"></i>
        </a>
        <a href="#" aria-label="TikTok">
          <i class="fab fa-tiktok"></i>
        </a>
        <a href="https://wa.me/212722582598"
          aria-label="WhatsApp">
          <i class="fab fa-whatsapp"></i>
        </a>
      </div>
    </div>
```

```
</div>
```

Cette partie crée la première section du footer contenant : le logo du site, une description de la boutique et les réseaux sociaux. Elle améliore l'identité visuelle et la communication avec les utilisateurs.

4.2 Liens rapides

Listing 4.2 – Liens rapides footer

```
<div class="footer-col">
  <h4>Quick Links</h4>
  <a href="index.php">Home</a>
  <a href="index.php#products">Products</a>
  <a href="index.php#bestsellers">Best Sellers</a>
  <a href="contact.php">Contact Us</a>
</div>
```

Cette section contient les liens rapides permettant aux utilisateurs de naviguer facilement entre les principales pages du site web.

4.3 Mon compte

Listing 4.3 – Section Mon compte dans le footer

```
<div class="footer-col">
  <h4>My Account</h4>
  <?php if (isset($_SESSION['user'])): ?>
    <a href="orders.php">My Orders</a>
    <a href="wishlist.php">My Wishlist</a>
    <a href="cart.php">My Cart</a>
    <a href="auth.php?action=logout">Logout</a>
  <?php else: ?>
    <a href="auth.php">Login</a>
    <a href="auth.php?tab=register">Create Account</a>
    <a href="cart.php">My Cart</a>
  <?php endif; ?>
</div>
```

Cette partie vérifie automatiquement si un utilisateur est connecté grâce aux sessions PHP. Le système affiche ensuite les options du compte utilisateur : les commandes, la wishlist, le panier et le bouton logout. Sinon, les boutons Login et Create Account apparaissent.

4.4 Informations de contact

Listing 4.4 – Informations contact footer

```
<div class="footer-col">
  <h4>Contact</h4>
  <p><i class="fas fa-map-marker-alt"></i>
    Fashion District, Oujda</p>
  <p><i class="fas fa-phone"></i> +212 722 582 598</p>
  <p><i class="fas fa-envelope"></i>
    m.foulko-224@ump.ac.ma</p>
</div>
```

Cette section affiche les informations de contact de la boutique afin de faciliter la communication avec les visiteurs du site web.

4.5 Copyright dynamique

Listing 4.5 – Copyright automatique

```
<div class="footer-bottom">
  <div class="container">
    <p>
      &copy; <?= date('Y') ?> M&C Fashion.
      All rights reserved.
    </p>
  </div>
</div>
```

Cette partie affiche automatiquement l'année actuelle dans le copyright du site grâce à la fonction PHP `date('Y')`.

4.6 Scripts JavaScript

Listing 4.6 – Import des scripts JavaScript

```
<script src="https://cdn.jsdelivr.net/npm/
  swiper@8/swiper-bundle.min.js"></script>
<script src="main.js"></script>
</body>
</html>
```

Cette partie importe les fichiers JavaScript utilisés dans le projet afin de gérer : les animations, le slider, les interactions dynamiques et les fonctionnalités du site web.

Chapitre 5

Fichier db.php

Le fichier `db.php` permet de connecter le site web à la base de données MySQL afin de gérer les produits, les utilisateurs et les informations du projet e-commerce.

Listing 5.1 – Connexion à la base de données — db.php

```
$host      = '127.0.0.1';
$dbname    = 'm&c_fashion';
$username  = 'root';
$password  = '';

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8",
        $username,
        $password
    );
    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );
    $pdo->setAttribute(
        PDO::ATTR_DEFAULT_FETCH_MODE,
        PDO::FETCH_ASSOC
    );
} catch (PDOException $e) {
    die("Erreur de connexion : " . $e->getMessage());
}
```

Ce fichier permet de créer une connexion sécurisée entre PHP et la base de données MySQL grâce à PDO. Le système utilise également `try/catch` pour gérer les erreurs de connexion et assurer le bon fonctionnement du site web.

Détail des paramètres :

- `$host` — adresse du serveur MySQL
- `$dbname` — nom de la base de données
- `$username` / `$password` — identifiants de connexion
- `charset=utf8` — gestion correcte des caractères spéciaux
- `ATTR_ERRMODE` — activation de l’affichage des erreurs PDO

- `ATTR_DEFAULT_FETCH_MODE` — récupération en tableau associatif
- `die()` — arrêt du script avec message d'erreur

Chapitre 6

Fichier auth.php

Le fichier `auth.php` permet de gérer le système d'authentification du site e-commerce. Il contrôle :

- la connexion utilisateur
- l'inscription
- la déconnexion
- les sessions PHP
- la sécurité des comptes utilisateurs

6.1 Initialisation et vérification de session

Listing 6.1 – Initialisation auth.php

```
include 'db.php';
session_start();

if (isset($_SESSION['user'])) {
    header("Location: index.php");
    exit;
}
```

Cette partie connecte la base de données MySQL et démarre les sessions PHP. Le système vérifie également si l'utilisateur est déjà connecté afin de le rediriger automatiquement vers la page d'accueil.

6.2 Déconnexion

Listing 6.2 – Gestion de la déconnexion

```
if (isset($_GET['action']) &&
```

```

$_GET['action'] === 'logout') {
    session_destroy();
    header("Location: auth.php?msg=logged_out");
    exit;
}

```

Cette partie permet de gérer la déconnexion utilisateur. Le système supprime les sessions PHP puis redirige automatiquement vers la page de connexion avec un message de confirmation.

6.3 Connexion utilisateur

Listing 6.3 – Gestion de la connexion

```

if (isset($_POST['login'])) {
    $username = trim($_POST['username']);
    $password = $_POST['password'];

    if (empty($username) || empty($password)) {
        $login_error = 'Please fill in all fields.';
    } else {
        $stmt = $pdo->prepare(
            "SELECT * FROM users WHERE username = ?"
        );
        $stmt->execute([$username]);
        $user = $stmt->fetch();

        if ($user &&
            password_verify($password, $user['password'])) {
            $_SESSION['user'] = $user['username'];
            $_SESSION['user_id'] = $user['id'];
            header("Location: index.php");
            exit;
        } else {
            $login_error = 'Incorrect username or password.';
        }
    }
}
}

```

Cette partie gère la connexion utilisateur. Le système récupère les informations saisies dans le formulaire puis vérifie :

- si les champs sont remplis
- si l'utilisateur existe dans MySQL
- si le mot de passe correspond au hash enregistré

En cas de succès, les informations utilisateur sont enregistrées dans les sessions PHP puis l'utilisateur est redirigé vers la page d'accueil.

6.4 Inscription utilisateur

Listing 6.4 – Gestion de l'inscription

```

if (isset($_POST['register'])) {
    $username = trim($_POST['username']);
    $email = trim($_POST['email']);
    $password = $_POST['password'];
    $password2 = $_POST['password2'];

    if (empty($username) || empty($email) || empty($password)) {
        $register_error = 'Please_fill_in_all_fields.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $register_error = 'Please_enter_a_valid_email.';
    } elseif (strlen($password) < 6) {
        $register_error = 'Password_min_6_characters.';
    } elseif ($password !== $password2) {
        $register_error = 'Passwords_do_not_match.';
    } else {
        $check = $pdo->prepare(
            "SELECT id FROM users
            WHERE username = ? OR email = ?"
        );
        $check->execute([$username, $email]);

        if ($check->rowCount() > 0) {
            $register_error = 'Username_or_email_taken.';
        } else {
            $hashed = password_hash(
                $password, PASSWORD_DEFAULT
            );
            $ins = $pdo->prepare(
                "INSERT INTO users (username, email, password)
                VALUES (?, ?, ?)"
            );
            $ins->execute([$username, $email, $hashed]);
            $_SESSION['user'] = $username;
            header("Location: index.php");
            exit;
        }
    }
}

```

Cette partie gère l'inscription utilisateur. Le système vérifie : les champs obligatoires, la validité de l'email, la longueur du mot de passe et la confirmation. Ensuite, le mot de passe est sécurisé grâce au hashage avant d'être enregistré dans MySQL. Après la création du compte, l'utilisateur est automatiquement connecté au site web.

6.5 Formulaire HTML

Listing 6.5 – Formulaire de connexion

```
<form id="form-login" action="auth.php" method="POST">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit" name="login">Sign In</button>
</form>
```

Cette partie crée le formulaire de connexion permettant aux utilisateurs d'accéder à leur compte dans le site e-commerce.

Listing 6.6 – Formulaire d'inscription

```
<form id="form-register" action="auth.php" method="POST">
  <input type="text" name="username">
  <input type="email" name="email">
  <input type="password" name="password">
  <input type="password" name="password2">
  <button type="submit" name="register">
    Create Account
  </button>
</form>
```

Cette partie crée le formulaire d'inscription permettant aux visiteurs de créer un nouveau compte utilisateur.

6.6 Changement d'onglet JavaScript

Listing 6.7 – Fonction switchTab()

```
function switchTab(tab) {
  document.querySelectorAll('.auth-tab')
    .forEach(function(t) {
      t.classList.toggle(
        'active',
        t.getAttribute('data-tab') === tab
      );
    });
  document.getElementById('form-login').style.display =
    (tab === 'login') ? 'block' : 'none';
  document.getElementById('form-register').style.display =
    (tab === 'register') ? 'block' : 'none';
}
```

Cette fonction JavaScript permet de changer dynamiquement entre les formulaires Login et Register sans recharger la page web.

Chapitre 7

Fichier cart.php

Le fichier `cart.php` permet de gérer le panier du site e-commerce. Il contrôle :

- l’affichage des produits sélectionnés
- la modification des quantités
- la suppression des produits
- le calcul du total
- la validation du panier avant la commande

7.1 Initialisation du panier

Listing 7.1 – Initialisation cart.php

```
include 'db.php';
session_start();

if (!isset($_SESSION['cart'])) $_SESSION['cart'] = [];
```

Cette partie connecte la base de données MySQL et démarre les sessions PHP. Le système vérifie également si le panier existe dans la session utilisateur. Si le panier n'existe pas, un tableau vide est automatiquement créé.

7.2 Augmenter la quantité

Listing 7.2 – Augmenter la quantité d’un article

```
if (isset($_GET['action']) &&
    $_GET['action'] === 'increase' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    foreach ($_SESSION['cart'] as &$item) {
```

```
        if ($item['id'] == $id) {
            $item['qty']++;
            break;
        }
    }
    unset($item);
    header("Location:␣cart.php");
    exit;
}
```

Cette partie permet d'augmenter automatiquement la quantité d'un produit dans le panier. Le système récupère l'identifiant produit depuis l'URL, recherche le produit dans la session panier, augmente la quantité et recharge automatiquement la page panier.

7.3 Diminuer la quantité

Listing 7.3 – Diminuer la quantité d'un article

```
if (isset($_GET['action']) &&
    $_GET['action'] === 'decrease' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    foreach ($_SESSION['cart'] as &$item) {
        if ($item['id'] == $id) {
            if ($item['qty'] > 1) $item['qty']--;
            break;
        }
    }
    unset($item);
    header("Location:␣cart.php");
    exit;
}
```

Cette partie permet de diminuer la quantité d'un produit dans le panier. Le système vérifie également que la quantité reste supérieure à 1 afin d'éviter une quantité invalide.

7.4 Supprimer un article

Listing 7.4 – Supprimer un article du panier

```
if (isset($_GET['action']) &&
    $_GET['action'] === 'remove' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    foreach ($_SESSION['cart'] as $key => $item) {
        if ($item['id'] == $id) {
```

```
        unset($_SESSION['cart'][$key]);  
        break;  
    }  
}  
$_SESSION['cart'] = array_values($_SESSION['cart']);  
header("Location:␣cart.php");  
exit;  
}
```

Cette partie permet de supprimer un produit du panier. Le système recherche le produit, le supprime de la session, réorganise automatiquement le tableau panier et recharge la page panier.

7.5 Vider le panier

Listing 7.5 – Vider complètement le panier

```
if (isset($_GET['action']) &&  
    $_GET['action'] === 'clear') {  
    $_SESSION['cart'] = [];  
    header("Location:␣cart.php");  
    exit;  
}
```

Cette partie permet de vider complètement le panier utilisateur. Tous les produits enregistrés dans la session sont supprimés automatiquement.

7.6 Calcul du total

Listing 7.6 – Calcul du total panier et frais de livraison

```
$cart_total = 0;  
$cart_count = 0;  
$shipping   = 0;  
  
foreach ($_SESSION['cart'] as $item) {  
    $cart_total += $item['price'] * $item['qty'];  
    $cart_count += $item['qty'];  
}  
  
if ($cart_total > 0 && $cart_total < 500) {  
    $shipping = 30;  
}
```


Cette partie calcule automatiquement : le prix total du panier, le nombre total des produits et les quantités des produits. Chaque sous-total est obtenu grâce à **prix x quantité**.

Si le total du panier est inférieur à 500 MAD, des frais de livraison sont ajoutés (30 MAD). Sinon, la livraison devient gratuite.

7.7 Affichage des articles

Listing 7.7 – Affichage des articles du panier

```
<?php if (empty($_SESSION['cart'])): ?>
    <!-- Message panier vide -->
<?php endif; ?>

<?php foreach ($_SESSION['cart'] as $item): ?>
    
    <?=$_number_format($item['price'], 2) ?> MAD
    <a href="cart.php?action=decrease&id=<?=$_$item['id']_?>">
    <a href="cart.php?action=increase&id=<?=$_$item['id']_?>">
    <a href="cart.php?action=remove&id=<?=$_$item['id']_?>"
        onclick="returnconfirm('Remove_this_item?')">
<?php endforeach; ?>

<?php if (isset($_SESSION['user'])): ?>
    <a href="checkout.php" class="btn_btn-full">
<?php endif; ?>
```

Ces éléments gèrent l’affichage complet du panier :

- `empty()` vérifie si le panier est vide
- la boucle `foreach` affiche automatiquement tous les produits
- `htmlspecialchars()` sécurise les données affichées
- `number_format()` formate correctement les prix
- le `confirm()` JavaScript affiche une confirmation avant suppression
- la vérification session protège l’accès au checkout

Chapitre 8

Fichier wishlist.php

Le fichier `wishlist.php` permet de gérer complètement le système des favoris du site e-commerce. Il permet aux utilisateurs de sauvegarder leurs produits préférés afin de les consulter plus tard ou les ajouter directement au panier.

Le système fonctionne de deux manières :

- pour les utilisateurs connectés : les favoris sont enregistrés dans MySQL
- pour les visiteurs invités : les favoris sont enregistrés dans les sessions PHP

8.1 Initialisation

Listing 8.1 – Initialisation wishlist.php

```
include 'db.php';
session_start();
if (!isset($_SESSION['wishlist']))
    $_SESSION['wishlist'] = [];
```

Cette première partie initialise le système wishlist. Le fichier connecte MySQL avec PHP, démarre les sessions PHP et crée automatiquement une wishlist vide si elle n'existe pas encore.

8.2 Suppression d'un favori

Listing 8.2 – Suppression d'un produit des favoris

```
if (isset($_GET['action']) &&
    $_GET['action'] === 'remove' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    if (isset($_SESSION['user_id'])) {
        $pdo->prepare(
            "DELETE FROM wishlist
```

```

        WHERE user_id = ? AND product_id = ?"
    )->execute([$SESSION['user_id'], $id]);

    $wRows = $pdo->prepare(
        "SELECT product_id FROM wishlist
        WHERE user_id = ?"
    );
    $wRows->execute([$SESSION['user_id']]);
    $SESSION['wishlist'] =
        array_column($wRows->fetchAll(), 'product_id');
} else {
    $SESSION['wishlist'] = array_values(
        array_diff($SESSION['wishlist'], [$id])
    );
}
$SESSION['flash'] = [
    'type' => 'info',
    'msg' => 'Item removed from your wishlist.'
];
header("Location: wishlist.php");
exit;
}

```

Cette grande partie gère complètement la suppression d'un produit favori. Le système récupère l'ID produit depuis l'URL, vérifie si l'utilisateur est connecté, supprime le produit depuis MySQL, met à jour automatiquement la session wishlist, affiche un message de confirmation et recharge automatiquement la page.

8.3 Ajout au panier depuis la wishlist

Listing 8.3 – Ajout au panier depuis les favoris

```

if (isset($_GET['action']) &&
    $_GET['action'] === 'add_to_cart' &&
    isset($_GET['id'])) {
    $id = intval($_GET['id']);
    $stmt = $pdo->prepare(
        "SELECT * FROM products WHERE id = ?"
    );
    $stmt->execute([$id]);
    $product = $stmt->fetch();

    if ($product) {
        if (!isset($SESSION['cart']))
            $SESSION['cart'] = [];
        $found = false;
        foreach ($SESSION['cart'] as &$item) {
            if ($item['id'] == $id) {
                $item['qty']++;
                $found = true;
                break;
            }
        }
    }
}

```

```

    }
    unset($item);
    if (!$found) {
        $product['qty'] = 1;
        $_SESSION['cart'][] = $product;
    }
    $_SESSION['flash'] = [
        'type' => 'success',
        'msg'  => $product['name'] . ' ajouté au panier !'
    ];
}
header("Location: wishlist.php");
exit;
}

```

Cette partie permet d'ajouter directement un produit favori au panier. Le système récupère les informations produit depuis MySQL, vérifie si le produit est déjà présent dans le panier, augmente automatiquement la quantité si oui, sinon ajoute un nouveau produit avec quantité = 1.

8.4 Récupération des produits favoris

Listing 8.4 – Récupération des produits favoris

```

$wishlist_products = [];
if (!empty($_SESSION['wishlist'])) {
    $ids = implode(',',
        array_map('intval', $_SESSION['wishlist'])
    );
    $wishlist_products =
        $pdo->query(
            "SELECT * FROM products WHERE id IN ($ids)"
        )->fetchAll();
}

```

Cette partie récupère automatiquement tous les produits favoris depuis la base de données. Le système récupère tous les IDs favoris, les transforme en format SQL et exécute une requête dynamique.

8.5 Synchronisation BDD/Session

Listing 8.5 – Synchronisation wishlist BDD et session

```

if (isset($_SESSION['user_id'])) {
    $wRows = $pdo->prepare(
        "SELECT product_id FROM wishlist WHERE user_id = ?"
    );
}

```

```

);
$rows->execute([$SESSION['user_id']]);
$SESSION['wishlist'] =
    array_column($rows->fetchAll(), 'product_id');
}

```

Cette partie synchronise automatiquement la wishlist entre MySQL et les sessions PHP. Ainsi, les favoris restent toujours à jour, même après actualisation de la page ou après reconnexion utilisateur.

8.6 Affichage des produits favoris

Listing 8.6 – Affichage des favoris

```

<?php if (isset($SESSION['flash'])): ?>
    <div class="flash-bar"><?=$f['type']>
        <?=$f['msg']>
    </div>
<?php endif; ?>

<?php if (empty($wishlist_products)): ?>
    <div class="empty-state">
        <h3>Your wishlist is empty</h3>
        <a href="index.php#products" class="btn">
            Explore Products
        </a>
    </div>
<?php endif; ?>

<?php foreach ($wishlist_products as $p): ?>
    <div class="product-card">
        
            <?=$p['price']> MAD
        </span>
        <a href="wishlist.php?action=add_to_cart&id=
        <?=$p['id']>
        <a href="wishlist.php?action=remove&id=
        <?=$p['id']>
    </div>
<?php endforeach; ?>

```

Cette partie gère l’affichage complet de la wishlist : les messages flash, les cartes produits (image, nom, prix), les boutons d’action. Le système utilise `foreach`, `htmlspecialchars` et `number_format` afin de sécuriser les données et formater correctement les prix.

Chapitre 9

Fichier product.php

Le fichier `product.php` permet d'afficher les détails complets d'un produit du site e-commerce. Il permet également :

- afficher les informations produit
- ajouter le produit au panier
- ajouter le produit aux favoris
- afficher les produits similaires
- vérifier le stock produit

9.1 Récupération et validation du produit

Listing 9.1 – Récupération du produit depuis MySQL

```
include 'db.php';
session_start();
if (!isset($_SESSION['cart']))
    $_SESSION['cart'] = [];
if (!isset($_SESSION['wishlist']))
    $_SESSION['wishlist'] = [];

$id = isset($_GET['id']) ? intval($_GET['id']) : 0;
if ($id === 0) {
    header("Location: index.php");
    exit;
}

$stmt = $pdo->prepare(
    "SELECT * FROM products WHERE id = ?"
);
$stmt->execute([$id]);
$product = $stmt->fetch();
if (!$product) {
    header("Location: index.php");
    exit;
}
```

Cette partie initialise le système produit, connecte MySQL avec PHP, démarre les sessions, crée le panier et la wishlist si nécessaire. L'ID produit est récupéré depuis l'URL et sécurisé avec `intval()`. Si aucun produit valide n'est trouvé, l'utilisateur est redirigé vers la page d'accueil.

9.2 Produits similaires et état wishlist

Listing 9.2 – Produits similaires et état wishlist

```
$similar = $pdo->prepare(
    "SELECT * FROM products
    WHERE category = ? AND id != ? LIMIT 4"
);
$similar->execute([$product['category'], $id]);
$similar_products = $similar->fetchAll();

$is_wished = in_array($id, $_SESSION['wishlist']);
$page_title = $product['name'];
include 'includes/header.php';
```

Cette partie récupère automatiquement les produits de la même catégorie (en excluant le produit actuel, limité à 4). Elle vérifie également si le produit est déjà dans les favoris et définit dynamiquement le titre de la page.

9.3 Affichage des détails produit

Listing 9.3 – Affichage des informations produit

```
<h1><?= htmlspecialchars($product['name']) ?></h1>
<p class="breadcrumb">
    <a href="index.php">Home</a> /
    <a href="index.php#products">Products</a> /
    <?= htmlspecialchars($product['name']) ?>
</p>

"

<?= ucfirst($product['category']) ?>
<?= number_format($product['price'], 2) ?> MAD
<?= htmlspecialchars($product['description']) ?>

<?= $product['stock'] > 0
    ? 'In stock(' . $product['stock'] . 'available)'
    : '<span style="color:red;">Out of stock</span>' ?>
```

Cette partie affiche dynamiquement : le nom, le breadcrumb, l'image, la catégorie, le prix formaté, la description et le statut stock du produit. Toutes les données sont sécurisées via `htmlspecialchars()`.

9.4 Boutons Add to Cart et Wishlist

Listing 9.4 – Boutons d'action produit

```
<?php if ($product['stock'] > 0): ?>
    <a href="index.php?action=add&id=<?=htmlspecialchars($product['id'])?"
        class="btn">
        <i class="fas fa-shopping-cart"></i> Add to Cart
    </a>
<?php endif; ?>

<a href="index.php?action=toggle_wishlist&id=
    <?=htmlspecialchars($product['id'])?"
    class="btn btn-outline-<?=htmlspecialchars($is_wished?'btn-gold':'')?">
    <i class="fas fa-heart"></i>
    <?= $is_wished ? 'Saved' : 'Save' ?>
</a>
```

Le bouton “Add to Cart” n’est affiché que si le produit est disponible en stock. Le bouton wishlist change dynamiquement de style (classe `btn-gold`) et de texte (Save / Saved) selon l’état du favori.

9.5 Produits similaires

Listing 9.5 – Affichage des produits similaires

```
<?php if (!empty($similar_products)): ?>
    <?php foreach ($similar_products as $p): ?>
        <div class="product-card">
            
            <h4><?= htmlspecialchars($p['name']) ?></h4>
            <span class="product-price">
                <?= number_format($p['price'], 2) ?> MAD
            </span>
            <a href="index.php?action=add&id=<?=htmlspecialchars($p['id'])?"
                class="overlay-btn">
                <i class="fas fa-shopping-cart"></i>
            </a>
            <a href="product.php?id=<?=htmlspecialchars($p['id'])?"
                class="overlay-btn">
                <i class="fas fa-eye"></i>
            </a>
```



```
</div>
<?php endforeach; ?>
<?php endif; ?>
```

Cette partie affiche automatiquement les produits similaires sous forme de cartes. Chaque carte contient l'image, le nom, le prix et des boutons pour ajouter au panier ou voir les détails.

Chapitre 10

Fichier checkout.php

Le fichier `checkout.php` permet de gérer la validation finale des commandes du site e-commerce. Il contrôle :

- la vérification utilisateur connecté
- la validation du panier
- le formulaire livraison
- le calcul total commande
- l'enregistrement commande dans MySQL
- l'enregistrement des produits commandés
- la confirmation commande

10.1 Protection de la page

Listing 10.1 – Vérification d'accès au checkout

```
include 'db.php';
session_start();

if (!isset($_SESSION['user'])) {
    $_SESSION['redirect_after_login'] = 'checkout.php';
    $_SESSION['flash'] = [
        'type' => 'info',
        'msg' => 'Please sign in to place your order.'
    ];
    header("Location: auth.php");
    exit;
}

if (empty($_SESSION['cart'])) {
    header("Location: cart.php");
    exit;
}
```

Cette partie protège automatiquement la page checkout. Le système vérifie si l'utilisateur est connecté, mémorise la page actuelle et redirige automatiquement vers la connexion. Si le panier est vide, l'utilisateur est redirigé vers le panier pour éviter les commandes invalides.

10.2 Calcul du total

Listing 10.2 – Calcul du total commande

```
$cart_total = 0;
$cart_count = 0;

foreach ($_SESSION['cart'] as $item) {
    $cart_total += $item['price'] * $item['qty'];
    $cart_count += $item['qty'];
}

$shipping = ($cart_total < 500) ? 30 : 0;
$grand_total = $cart_total + $shipping;
```

Cette partie calcule automatiquement : le prix total panier, le nombre total produits, les frais livraison (30 MAD si total inférieur à 500 MAD, sinon gratuit) et le total général commande.

10.3 Traitement de la commande

Listing 10.3 – Enregistrement de la commande dans MySQL

```
if (isset($_POST['place_order'])) {
    $name = trim($_POST['name']);
    $email = trim($_POST['email']);
    $address = trim($_POST['address']);
    $city = trim($_POST['city']);

    if (empty($name) || empty($email) ||
        empty($address) || empty($city)) {
        $order_error = 'Please fill in all required fields.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $order_error = 'Please enter a valid email.';
    }

    $full_address = "$address, $city";
    $ins = $pdo->prepare(
        "INSERT INTO orders
        (user_id, customer_name, customer_email,
        customer_address, total_amount)
        VALUES (?, ?, ?, ?, ?)"
```

```

    );
    $ins->execute([
        $_SESSION['user_id'], $name, $email,
        $full_address, $grand_total
    ]);
    $order_id = $pdo->lastInsertId();

    foreach ($_SESSION['cart'] as $item) {
        $insItem = $pdo->prepare(
            "INSERT INTO order_items
            (order_id, product_id, product_name,
            product_price, quantity)
            VALUES (?, ?, ?, ?, ?)"
        );
        $insItem->execute([
            $order_id, $item['id'], $item['name'],
            $item['price'], $item['qty']
        ]);
    }

    $_SESSION['cart'] = [];
    $order_success = true;
    $order_number = str_pad($order_id, 6, '0', STR_PAD_LEFT);
}

```

Cette partie enregistre automatiquement la commande principale dans MySQL puis enregistre tous les produits commandés en les liant à l'ID commande via `lastInsertId()`. Le panier est ensuite vidé et un numéro commande formaté est généré (ex. : 000125).

10.4 Affichage de la confirmation

Listing 10.4 – Confirmation de commande

```

<?php if ($order_success): ?>
    <h2>Order Confirmed!</h2>
    <p>Thank you. Your order has been placed successfully.</p>
    <a href="orders.php" class="btn">View My Orders</a>
    <a href="index.php" class="btn btn-outline">
        Continue Shopping
    </a>
<?php endif; ?>

<?php if ($order_error): ?>
    <div class="alert alert-error">
        <?= htmlspecialchars($order_error) ?>
    </div>
<?php endif; ?>

```

Ces blocs conditionnels affichent la confirmation après validation commande (avec boutons vers les commandes ou pour continuer le shopping) ainsi que les messages d'erreur si les données sont invalides.

10.5 Formulaire checkout

Listing 10.5 – Formulaire de livraison

```
<form action="checkout.php" method="POST">
  <input type="text"   name="name"   class="form-input" required>
  <input type="email"  name="email"  class="form-input" required>
  <input type="tel"    name="phone"  class="form-input">

  <div class="checkout-section">
    <h3>Payment Method</h3>
    <strong>Cash on Delivery</strong>
  </div>

  <?php foreach ($_SESSION['cart'] as $item): ?>
    
    <p><?= htmlspecialchars($item['name']) ?></p>
    <?= number_format($item['price']*$item['qty'],2) ?> MAD
  <?php endforeach; ?>

  <button type="submit" name="place_order" class="btn btn-full">
    <i class="fas fa-lock"></i> Place Order
  </button>
</form>
```

Le formulaire checkout collecte les informations de livraison, affiche la méthode de paiement (Cash on Delivery), le résumé commande avec chaque produit, et le bouton de confirmation.

Chapitre 11

Fichier orders.php

Le fichier `orders.php` permet de gérer l'historique des commandes du site e-commerce. Il permet aux utilisateurs connectés :

- consulter leurs commandes
- afficher les détails commande
- voir les produits commandés
- vérifier le statut commande
- afficher les informations livraison

11.1 Protection et récupération des commandes

Listing 11.1 – Récupération et protection des commandes

```
include 'db.php';
session_start();

if (!isset($_SESSION['user'])) {
    $_SESSION['redirect_after_login'] = 'orders.php';
    header("Location: auth.php");
    exit;
}

$order_detail = null;
$order_items_detail = [];

if (isset($_GET['order_id'])) {
    $oid = intval($_GET['order_id']);
    $stmt0 = $pdo->prepare(
        "SELECT * FROM orders WHERE id=? AND user_id=?"
    );
    $stmt0->execute([$oid, $_SESSION['user_id']]);
    $order_detail = $stmt0->fetch();

    if ($order_detail) {
        $stmt1 = $pdo->prepare(
            "SELECT * FROM order_items WHERE order_id=?"
        );
    }
}
```

```

    );
    $stmtI->execute([$oid]);
    $order_items_detail = $stmtI->fetchAll();
}
}

$stmtOrders = $pdo->prepare(
    "SELECT * FROM orders WHERE user_id = ?
    ORDER BY created_at DESC"
);
$stmtOrders->execute([$SESSION['user_id']]);
$orders = $stmtOrders->fetchAll();

```

Cette partie protège la page commandes (redirection si non connecté), récupère les détails d'une commande spécifique si un `order_id` est présent dans l'URL (en vérifiant que la commande appartient bien à l'utilisateur), et récupère toutes les commandes triées du plus récent au plus ancien.

11.2 Affichage de la liste des commandes

Listing 11.2 – Liste des commandes

```

<?php if (empty($orders)): ?>
    <div class="empty-state">
        <h3>No orders yet</h3>
        <a href="index.php#products" class="btn">Shop Now</a>
    </div>
<?php endif; ?>

<?php foreach ($orders as $order): ?>
    <?php
        $countStmt = $pdo->prepare(
            "SELECT SUM(quantity) FROM order_items
            WHERE order_id = ?"
        );
        $countStmt->execute([$order['id']]);
        $item_count = $countStmt->fetchColumn();
    ?>
    <td><strong>
        #<? str_pad($order['id'], 6, '0', STR_PAD_LEFT) ?>
    </strong></td>
    <td><? date('d_M_Y', strtotime($order['created_at'])) ?></td>
    <td><? $item_count ?> item<? $item_count>1?'s':'' ?></td>
    <td>
        <span class="status-badge status-<?=$order['status']_?>">
            <? ucfirst($order['status']) ?>
        </span>
    </td>
    <td>
        <a href="orders.php?order_id=<?=$order['id']_?>"
            class="btn btn-outline btn-sm">View</a>
    </td>

```

```
<?php endforeach; ?>
```

Cette partie affiche la liste complète des commandes avec : numéro formaté, date lisible, nombre d'articles, badge de statut coloré et bouton pour voir les détails.

11.3 Modal détails commande

Listing 11.3 – Modal détails d'une commande

```
<?php if ($order_detail): ?>
  <div class="modal-overlay_open">
    <div class="modal_modal-lg">
      <?= htmlspecialchars($order_detail['customer_name']) ?>
      <?= htmlspecialchars($order_detail['customer_email']) ?>
      <?= htmlspecialchars($order_detail['customer_address']) ?>
      <?= date('d_F_Y, H:i',
        strtotime($order_detail['created_at'])) ?>

      <?php foreach ($order_items_detail as $item): ?>
        <strong>
          <?= htmlspecialchars($item['product_name']) ?>
        </strong>
        <small>
          Qty: <?= $item['quantity'] ?> x
          <?= number_format($item['product_price'],2) ?> MAD
        </small>
        <?= number_format(
          $item['product_price']*$item['quantity'],2) ?> MAD
      <?php endforeach; ?>

      Total:
      <?= number_format($order_detail['total_amount'],2) ?> MAD

      <button onclick="closeModal('modalOrderDetail')">
        Close
      </button>
      <a href="orders.php" class="btn btn-outline btn-sm">
        Back to Orders
      </a>
    </div>
  </div>
<?php endif; ?>
```

Cette partie affiche automatiquement une fenêtre modal contenant tous les détails de la commande : informations client, date complète avec heure, liste des produits avec quantités et prix unitaires, montant total.

Chapitre 12

Fichier contact.php

Le fichier `contact.php` permet de gérer le formulaire de contact du site e-commerce. Il permet aux visiteurs :

- envoyer des messages
- contacter le support
- sauvegarder les messages dans MySQL
- afficher les informations contact
- afficher les messages erreurs ou succès

12.1 Traitement du formulaire

Listing 12.1 – Traitement du formulaire contact

```
include 'db.php';
session_start();
$success = false;
$error = '';

if (isset($_POST['send_contact'])) {
    $name = trim($_POST['name']);
    $email = trim($_POST['email']);
    $message = trim($_POST['message']);

    if (empty($name) || empty($email) || empty($message)) {
        $error = 'Please fill in all fields.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = 'Please enter a valid email address.';
    } else {
        $stmt = $pdo->prepare(
            "INSERT INTO contact_messages
            (name, email, message)
            VALUES (?, ?, ?)"
        );
        $stmt->execute([$name, $email, $message]);
        $success = true;
    }
}
```

}

Cette grande partie gère complètement le formulaire contact. Le système récupère les informations utilisateur, supprime les espaces inutiles, vérifie les champs obligatoires, contrôle le format email, prépare une requête SQL sécurisée, enregistre le message dans MySQL et active automatiquement le message succès.

12.2 Informations de contact et formulaire

Listing 12.2 – Affichage des informations contact

```
<div class="contact-item">
  <i class="fas fa-map-marker-alt"></i>
  <p>Fashion District, Oujda, Morocco</p>
</div>
<div class="contact-item">
  <i class="fas fa-phone"></i>
  <p>+212 722 582 598</p>
</div>
<div class="contact-item">
  <i class="fas fa-envelope"></i>
  <p>m.foulko-224@ump.ac.ma</p>
</div>
<div class="contact-item">
  <i class="fab fa-whatsapp"></i>
  <a href="https://wa.me/212722582598">Chat with us</a>
</div>
<div>
  <h4>Opening Hours</h4>
  <p>Monday - Friday: 9:00 - 18:00</p>
  <p>Saturday: 10:00 - 15:00</p>
</div>
```

Cette partie affiche les informations de contact : adresse, téléphone, email, lien WhatsApp et horaires d'ouverture.

Listing 12.3 – Formulaire de contact et messages

```
<?php if ($success): ?>
  <div>
    <h3>Message Sent!</h3>
    <p>Thank you for reaching out. We will get back
      to you soon.</p>
    <a href="contact.php" class="btn btn-outline">
      Send Another Message
    </a>
  </div>
<?php else: ?>
  <?php if ($error): ?>
    <div class="alert alert-error">
```

```
<?= htmlspecialchars($error) ?>
</div>
<?php endif; ?>

<form action="contact.php" method="POST">
  <input type="text" name="name" class="form-input"
    required>
  <input type="email" name="email" class="form-input"
    required>
  <input type="text" name="subject" class="form-input">
  <textarea name="message" class="form-input" rows="6"
    required></textarea>
  <button type="submit" name="send_contact"
    class="btn btn-full">
    <i class="fas fa-paper-plane"></i> Send Message
  </button>
</form>
<?php endif; ?>
```

Le formulaire inclut : nom, email, sujet et zone de message. Après envoi réussi, un message de confirmation s'affiche avec un bouton pour envoyer un nouveau message.

Chapitre 13

Fichier style.css

Le fichier `style.css` est la feuille de style principale du site e-commerce M&C Fashion. Son rôle principal est de transformer les pages HTML simples en une interface moderne, élégante, responsive et professionnelle.

Grâce au CSS :

- le site possède une identité visuelle cohérente
- les pages deviennent interactives
- les animations améliorent l'expérience utilisateur
- le design s'adapte automatiquement aux téléphones et ordinateurs

13.1 Variables globales CSS

Listing 13.1 – Variables CSS globales

```
:root {  
  --green: #1a3d28;  
  --gold: #b8943f;  
  --bg: #f7f5f2;  
  --shadow: 0 4px 20px rgba(0,0,0,0.08);  
}
```

Cette partie crée des variables globales CSS réutilisables dans tout le site. Les variables permettent : centraliser les couleurs, réutiliser les mêmes styles et modifier facilement le thème du site. `--green` représente la couleur principale, `--gold` représente la couleur luxe du site.

13.2 Reset et styles de base

Listing 13.2 – Reset CSS et style body

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-body);
  background: var(--bg);
  color: var(--dark);
  line-height: 1.7;
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 24px;
}
```

Ces règles réinitialisent tous les styles par défaut du navigateur (marges, espaces) pour garantir un affichage identique. Le `body` définit la police, la couleur, le fond et l'interligne. Le `.container` centre le contenu et limite sa largeur à 1200px.

13.3 Boutons

Listing 13.3 – Styles des boutons

```
.btn {
  padding: 12px 32px;
  background: var(--green);
  border-radius: var(--radius);
  transition: var(--transition);
}

.btn:hover {
  background: var(--gold);
  transform: translateY(-2px);
}
```

Les boutons utilisent des couleurs modernes, des coins arrondis et des animations fluides. Au survol, le bouton change de couleur et monte légèrement grâce à `translateY()`.

13.4 Header

Listing 13.4 – Styles du header

```

.site-header {
  position:    fixed;
  top:         0;
  z-index:     1000;
  background:  rgba(255,255,255,0.96);
}

.header-inner {
  display:      flex;
  align-items:  center;
  justify-content: space-between;
}

.main-nav { display: flex; gap: 32px; }
.main-nav a:hover::after { width: 100%; }
.icon-btn { width: 40px; height: 40px; border-radius: 50%; }
.badge { position: absolute; background: var(--gold);
        border-radius: 50%; }
.user-dropdown { display: none; position: absolute; }
.user-menu:hover .user-dropdown { display: block; }

```

Le header est fixé en haut écran (`position:fixed`), reste visible pendant le scroll et passe au-dessus des autres éléments (`z-index:1000`). Flexbox organise le logo, la navigation et les boutons. Les badges s'affichent au-dessus des icônes avec `position:absolute`.

13.5 Hero Slider

Listing 13.5 – Styles du hero slider

```

.hero { height: 88vh; overflow: hidden; }
.hero-slide.active { opacity: 1; }

@keyframes heroIn {
  from { opacity: 0; transform: translateY(30px); }
  to   { opacity: 1; transform: translateY(0); }
}

```

Le hero occupe 88% de la hauteur écran. L'animation `heroIn` fait apparaître progressivement et monter doucement le contenu pour un effet moderne et professionnel.

13.6 Grille produits et interactions

Listing 13.6 – Grille et cartes produits

```
.products-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(260px, 1fr));
}

.product-card:hover {
  transform: translateY(-6px);
  box-shadow: var(--shadow-lg);
}

.product-overlay { opacity: 0; }
.product-card:hover .product-overlay { opacity: 1; }
```

CSS Grid crée automatiquement une grille responsive. Au survol, les cartes montent légèrement et révèlent les boutons rapides (panier, wishlist, détail) superposés à l'image.

13.7 Autres styles

Listing 13.7 – Cart, formulaires, modals, footer, responsive

```
.cart-layout {
  display: grid;
  grid-template-columns: 1fr 340px;
}

.form-input:focus { border-color: var(--green); }

.alert-error { background: #fdf0f0; color: #c0392b; }
.alert-success { background: #eafaf1; color: #1d6a3e; }

.modal-overlay {
  position: fixed;
  background: rgba(0,0,0,0.55);
}

.modal { max-width: 650px; border-radius: 10px; }

.flash-bar { position: fixed; top: var(--header-h); }

.footer-grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr 1.5fr;
}

@media (max-width: 768px) {
  /* Adaptation mobile */
}
```

Ces règles gèrent : la mise en page du panier (2 colonnes), le focus sur les formulaires, les couleurs des alertes (rouge/vert), les modals popup avec fond sombre, les messages flash, le footer en 4 colonnes et l'adaptation responsive mobile via @media.

Chapitre 14

Fichier main.js

Le fichier `main.js` est le script JavaScript principal du site e-commerce M&C Fashion. Son rôle est de rendre le site dynamique, interactif et moderne.

Grâce au JavaScript :

- le menu mobile devient interactif
- le slider fonctionne automatiquement
- les filtres produits réagissent sans recharger la page
- les modals s'ouvrent dynamiquement
- les animations améliorent l'expérience utilisateur
- les messages disparaissent automatiquement

14.1 Scroll et menu mobile

Listing 14.1 – Détection du scroll et menu hamburger

```
document.addEventListener('DOMContentLoaded', function () {

    const header = document.getElementById('site-header');

    window.addEventListener('scroll', function () {
        header.classList.toggle(
            'scrolled', window.pageY > 20
        );
    });

    const menuToggle =
        document.getElementById('menu-toggle');
    const mainNav =
        document.getElementById('main-nav');

    menuToggle.addEventListener('click', function () {
        mainNav.classList.toggle('open');
    });

    document.addEventListener('click', function (e) {
        if (!header.contains(e.target)) {
```

```
        mainNav.classList.remove('open');
    }
});

});
```

Le code attend le chargement complet du DOM (`DOMContentLoaded`). La détection du scroll ajoute/supprime la classe `scrolled` au header. Le menu hamburger s'ouvre/ferme au clic et se ferme automatiquement quand l'utilisateur clique en dehors.

14.2 Slider automatique

Listing 14.2 – Slider hero automatique

```
const heroSlides =
    document.querySelectorAll('.hero-slide');
const heroDots =
    document.querySelectorAll('.slider-dot');
let current = 0, sliderTimer;

function goTo(n) {
    heroSlides[current].classList.remove('active');
    current = (n + heroSlides.length) % heroSlides.length;
    heroSlides[current].classList.add('active');
}

function startAuto() {
    sliderTimer = setInterval(function () {
        goTo(current + 1);
    }, 5000);
}

function resetAuto() {
    clearInterval(sliderTimer);
    startAuto();
}

nextBtn.addEventListener('click', function () {
    goTo(current + 1);
    resetAuto();
});

heroDots.forEach(function (dot, i) {
    dot.addEventListener('click', function () {
        goTo(i);
    });
});
```

Le slider change automatiquement toutes les 5 secondes grâce à `setInterval()`. La fonction `goTo(n)` gère la transition entre slides avec une boucle infinie. `resetAuto()` redémarre le timer après une interaction manuelle.

14.3 Filtres produits

Listing 14.3 – Filtrage dynamique des produits

```
const filterBtns =
  document.querySelectorAll('.filter-btn');
const productCards =
  document.querySelectorAll('.product-card[data-cat]');

filterBtns.forEach(function (btn) {
  btn.addEventListener('click', function () {
    const cat = this.getAttribute('data-filter');
    productCards.forEach(function (card) {
      if (cat === 'all' ||
        card.getAttribute('data-cat') === cat) {
        card.style.display = 'block';
      } else {
        card.style.display = 'none';
      }
    });
  });
});
```

Le filtrage se fait instantanément sans recharger la page. Le système récupère la catégorie du bouton cliqué et affiche/masque les cartes produits correspondantes via `style.display`.

14.4 Système de modals

Listing 14.4 – Gestion des modals popup

```
function openModal(id) {
  const el = document.getElementById(id);
  if (el) el.classList.add('open');
}

function closeModal(id) {
  const el = document.getElementById(id);
  if (el) el.classList.remove('open');
}

document.addEventListener('keydown', function (e) {
  if (e.key === 'Escape') {
    document.querySelectorAll('.modal-overlay.open')
      .forEach(function(m) {
        m.classList.remove('open');
      });
  }
});
```

Les fonctions `openModal()` et `closeModal()` ajoutent/suppriment dynamiquement la classe CSS `open`. La touche Escape ferme toutes les modals ouvertes, améliorant l'accessibilité et le confort utilisateur.

14.5 Messages flash et onglets auth

Listing 14.5 – Messages flash et système d'onglets

```
const flashBar = document.querySelector('.flash-bar');
if (flashBar) {
  setTimeout(function () {
    flashBar.style.opacity = '0';
    setTimeout(function() {
      flashBar.remove();
    }, 500);
  }, 4000);
}

const authTabs = document.querySelectorAll('.auth-tab');
const authForms = document.querySelectorAll('.auth-form');

authTabs.forEach(function (tab) {
  tab.addEventListener('click', function () {
    const target = this.getAttribute('data-tab');
    authForms.forEach(function (form) {
      form.style.display =
        (form.id === 'form-' + target)
          ? 'block' : 'none';
    });
  });
});
```

Les messages flash disparaissent automatiquement après 4 secondes avec un effet de fondu, puis sont supprimés du DOM (`remove()`) pour optimiser les performances. Le système d'onglets login/register affiche/masque dynamiquement les formulaires sans rechargement de page.

Fonctionnement Global du Projet M&C; Fashion

Le projet fonctionne comme un système connecté où chaque fichier possède un rôle précis. Tous les composants communiquent entre eux via PHP, MySQL et les sessions utilisateur.

Fichier	Rôle principal
index.php	Affiche les produits principaux de la boutique
product.php	Affiche les détails complets d'un produit
cart.php	Gère le panier utilisateur (ajout, suppression, quantité)
wishlist.php	Gère les favoris (connexion MySQL ou session invité)
auth.php	Gère la connexion, l'inscription et la déconnexion
checkout.php	Valide et enregistre les commandes dans MySQL
orders.php	Affiche l'historique des commandes utilisateur
contact.php	Permet aux visiteurs de contacter le site
db.php	Connecte PHP avec MySQL via PDO
header.php	Structure commune : navigation, panier, wishlist
footer.php	Structure commune : liens, réseaux sociaux, contact
style.css	Design complet et responsive de toutes les pages
main.js	Animations, slider, filtres, modals et interactions

Concepts et Fonctions Techniques Utilisés dans le Projet

Cette section présente l'explication détaillée de chaque fonction, commande PHP, JavaScript et CSS utilisée dans le projet M&C; Fashion.

Fonctions et Commandes — index.php

<code>include()</code>	Inclure un autre fichier PHP dans la page actuelle.
<code>session_start()</code>	Démarrer les sessions PHP pour gérer les données utilisateur.
<code>isset()</code>	Vérifier si une variable ou une session existe.
<code>\$_SESSION</code>	Stocker temporairement les données utilisateur.
<code>intval()</code>	Transformer une valeur en nombre entier pour sécuriser l'identifiant produit.
<code>prepare()</code>	Préparer une requête SQL sécurisée avant son exécution.
<code>fetch()</code>	Récupérer une seule ligne depuis la base de données.
<code>fetchAll()</code>	Récupérer toutes les données sous forme de tableau.
<code>foreach()</code>	Parcourir plusieurs éléments automatiquement.
<code>in_array()</code>	Vérifier si une valeur existe dans un tableau.
<code>array_diff()</code>	Supprimer une valeur spécifique d'un tableau.
<code>array_column()</code>	Récupérer une colonne spécifique depuis un tableau de données.
<code>header()</code>	Rediriger l'utilisateur vers une autre page.
<code>htmlspecialchars()</code>	Sécuriser les données affichées contre les injections HTML.
<code>number_format()</code>	Formater correctement les prix et les nombres.
<code>empty()</code>	Vérifier si une variable est vide.

Fonctions et Commandes — header.php

<code>session_start()</code>	Démarrer les sessions PHP pour gérer les données utilisateur.
<code>isset()</code>	Vérifier si une variable ou une session existe.
<code>\$_SESSION</code>	Stocker temporairement les données utilisateur.
<code>foreach()</code>	Parcourir plusieurs éléments automatiquement.
<code>prepare()</code>	Préparer une requête SQL sécurisée avant son exécution.
<code>fetchColumn()</code>	Récupérer une seule valeur depuis MySQL.
<code>basename()</code>	Récupérer le nom du fichier actuel.
<code>\$_SERVER['PHP_SELF']</code>	Récupérer le nom de la page actuelle.
<code>htmlspecialchars()</code>	Sécuriser les données affichées contre les injections HTML.
<code><?= ?></code>	Afficher rapidement une variable PHP dans le HTML.
<code>+=</code>	Ajouter une valeur à une variable existante.
<code><header></code>	Créer l'en-tête principal du site.
<code><nav></code>	Créer la barre de navigation principale.

<code></code>	Afficher du texte ou des badges dans le header.
<code><i></code>	Afficher les icônes Font Awesome.
Google Fonts	Importer des polices personnalisées pour le design.
Font Awesome	Ajouter les icônes panier, utilisateur et favoris.
Responsive Design	Adapter le header aux téléphones et tablettes.
<code>aria-label=""</code>	Améliorer l'accessibilité du site web.

Fonctions et Commandes — footer.php

<code>htmlspecialchars()</code>	Sécuriser les données affichées dans le footer.
<code><footer></code>	Créer le bas de page principal du site.
<code><div></code>	Organiser les sections du footer.
<code><a href></code>	Créer des liens vers les pages et réseaux sociaux.
<code><i></code>	Afficher les icônes Font Awesome dans le footer.
<code>target="_blank"</code>	Ouvrir les liens externes dans un nouvel onglet.
<code>&copy;</code>	Afficher le symbole copyright dans le footer.
<code>date('Y')</code>	Afficher dynamiquement l'année actuelle dans le copyright.
Font Awesome	Afficher les icônes réseaux sociaux et contact.
CSS Flexbox/Grid	Organiser les éléments du footer de manière responsive.
Responsive Design	Adapter le footer aux téléphones et tablettes.

Fonctions et Commandes — db.php

PDO	Permettre la connexion entre PHP et MySQL.
<code>try { }</code>	Tester les erreurs possibles pendant la connexion.
<code>catch()</code>	Gérer les erreurs de connexion MySQL.
PDOException	Représenter les erreurs liées à PDO et MySQL.
<code>setAttribute()</code>	Configurer les paramètres de la connexion PDO.
<code>PDO::ATTR_ERRMODE</code>	Activer le mode gestion des erreurs PDO.
<code>PDO::ERRMODE_EXCEPTION</code>	Afficher les erreurs MySQL sous forme d'exception.
<code>charset=utf8mb4</code>	Gérer correctement les caractères spéciaux et accents.
<code>die()</code>	Arrêter le script en cas d'erreur critique.
<code>require_once()</code>	Inclure le fichier de connexion une seule fois dans le projet.
<code>prepare()</code>	Préparer des requêtes SQL sécurisées.

fetch()	Récupérer une ligne depuis MySQL.
fetchAll()	Récupérer plusieurs lignes depuis MySQL.
fetchColumn()	Récupérer une seule valeur depuis MySQL.

Fonctions et Commandes — auth.php

<code>trim()</code>	Supprimer les espaces inutiles avant et après les données saisies.
<code>filter_var()</code>	Vérifier et filtrer les données utilisateur.
<code>FILTER_VALIDATE_EMAIL</code>	Vérifier si l'adresse email respecte le bon format.
<code>password_hash()</code>	Crypter et sécuriser le mot de passe avant stockage MySQL.
<code>password_verify()</code>	Comparer le mot de passe saisi avec le hash enregistré.
<code>\$_POST</code>	Récupérer les informations envoyées depuis le formulaire.
<code>placeholder=""</code>	Afficher un texte indicatif dans les champs formulaire.
<code>required</code>	Empêcher l'envoi du formulaire si un champ obligatoire est vide.
<code>session_destroy()</code>	Déconnecter l'utilisateur et supprimer toutes les sessions.
<code>\$_SESSION['user']</code>	Stocker temporairement les informations utilisateur connecté.
<code>error message</code>	Afficher les messages d'erreur si les informations sont incorrectes.
<code>success message</code>	Afficher un message de succès après connexion ou inscription.

Fonctions et Commandes — cart.php

<code>array_key_exists()</code>	Vérifier si un produit existe déjà dans le panier.
<code>unset()</code>	Supprimer complètement un produit du panier ou une variable session.
<code>array_sum()</code>	Calculer automatiquement le total des prix ou des quantités.
<code>max()</code>	Empêcher une quantité inférieure à une valeur minimale autorisée.
<code>min()</code>	Limiter automatiquement une quantité maximale autorisée.
<code>\$_GET</code>	Récupérer les informations envoyées dans l'URL (identifiant produit).
<code>array_values()</code>	Réindexer le tableau panier après suppression.
<code>type="number"</code>	Permettre de saisir uniquement des valeurs numériques.
<code>colspan</code>	Fusionner plusieurs colonnes dans un tableau HTML.
<code>btn-remove</code>	Bouton utilisé pour supprimer un produit du panier.
<code>btn-checkout</code>	Bouton permettant d'accéder à la validation de commande.

Fonctions et Commandes — wishlist.php

<code>array_diff()</code>	Supprimer automatiquement une valeur depuis un tableau.
<code>array_values()</code>	Réorganiser les index du tableau après suppression.
<code>array_column()</code>	Récupérer uniquement une colonne depuis les résultats MySQL.
<code>implode()</code>	Transformer un tableau en chaîne de caractères compatible SQL.

<code>array_map()</code>	Appliquer une fonction sur chaque élément d'un tableau.
<code>query()</code>	Exécuter directement une requête SQL simple.
<code>fetchAll()</code>	Récupérer plusieurs lignes depuis MySQL sous forme tableau.
<code>\$_SESSION['flash']</code>	Créer des messages temporaires après certaines actions.
<code>foreach (&\$item)</code>	Modifier directement les éléments originaux du tableau.
<code>intval()</code>	Sécuriser les identifiants produits avant utilisation SQL.
<code>htmlspecialchars()</code>	Sécuriser les données affichées contre les injections HTML.
<code>number_format()</code>	Formater correctement les prix produits.
<code>empty()</code>	Vérifier si la wishlist est vide.
<code>confirm()</code>	Afficher une fenêtre JavaScript de confirmation avant suppression.

Fonctions et Commandes — product.php

ucfirst()	Mettre automatiquement la première lettre d'un texte en majuscule.
LIMIT	Limiter le nombre de produits similaires récupérés depuis MySQL.
id != ?	Exclure automatiquement le produit actuel des produits similaires.
breadcrumb	Afficher le chemin de navigation utilisateur dans le site web.
object-fit:cover	Adapter automatiquement l'image produit sans déformation.
loading="lazy"	Charger les images uniquement quand elles deviennent visibles.
btn-gold	Changer dynamiquement le style bouton wishlist si produit enregistré.
overlay-btn	Boutons interactifs affichés au survol de l'image produit.
page_title	Définir dynamiquement le titre de la page produit.
similar_products	Tableau contenant les produits similaires récupérés depuis MySQL.
in_array()	Vérifier si le produit est déjà dans la wishlist utilisateur.
intval()	Sécuriser l'ID produit reçu depuis l'URL.

Fonctions et Commandes — checkout.php

lastInsertId()	Récupérer l'identifiant de la dernière commande enregistrée.
str_pad()	Ajouter des zéros pour formater le numéro commande.
STR_PAD_LEFT	Ajouter les zéros au début du numéro commande (ex: 000125).
type="tel"	Créer un champ spécialement adapté aux numéros de téléphone.
redirect_after_login	Mémoriser la page checkout avant connexion utilisateur.
grand_total	Variable contenant le montant total final de la commande.
\$shipping	Variable contenant les frais de livraison (30 MAD si total < 500 MAD).
order_success	Booléen vérifiant si la commande a été validée avec succès.
order_error	Variable stockant les messages d'erreur de validation commande.
place_order	Bouton de soumission pour confirmer et enregistrer la commande.
Cash on Delivery	Méthode de paiement à la livraison utilisée dans le site.
fa-lock	Icône sécurité utilisée dans le bouton de validation commande.
filter_var()	Valider l'adresse email du client avant enregistrement.

Fonctions et Commandes — orders.php

ORDER BY created_at DESC	Trier les commandes des plus récentes aux plus anciennes.
fetchColumn()	Récupérer directement le total quantité produits depuis MySQL.

<code>date()</code>	Transformer automatiquement une date MySQL en format lisible utilisateur.
<code>strtotime()</code>	Convertir une date MySQL en timestamp PHP.
<code>str_pad()</code>	Ajouter des zéros avant le numéro commande pour le formatage.
<code>STR_PAD_LEFT</code>	Ajouter les zéros au début du texte.
<code>ucfirst()</code>	Mettre automatiquement la première lettre en majuscule.
<code>modal-overlay</code>	Créer une fenêtre popup affichant les détails commande.
<code>modal-lg</code>	Créer une grande fenêtre modal.
<code>status-badge</code>	Afficher visuellement le statut de la commande.
<code>closeModal()</code>	Fermer automatiquement la fenêtre détails commande.
<code>date('d M Y')</code>	Afficher la date au format lisible ex: 24 May 2026.
<code>date('d F Y, H:i')</code>	Afficher date complète avec heure et minute.
<code>item/items</code>	Gérer automatiquement le pluriel selon la quantité articles.
<code>AND user_id=?</code>	Sécuriser : vérifier que la commande appartient à l'utilisateur.

Fonctions et Commandes — contact.php

<code>trim()</code>	Supprimer automatiquement les espaces inutiles d'un texte saisi.
<code>filter_var()</code>	Vérifier si une donnée respecte un format spécifique.
<code>FILTER_VALIDATE_EMAIL</code>	Vérifier si l'email utilisateur possède un format valide.
<code><textarea></code>	Créer une zone texte permettant d'écrire des messages longs.
<code>rows="6"</code>	Définir automatiquement la hauteur du textarea.
<code>resize:vertical</code>	Permettre de modifier uniquement la hauteur du textarea.
<code>fa-paper-plane</code>	Icône utilisée dans le bouton d'envoi de message.
<code>fa-map-marker-alt</code>	Icône représentant l'adresse du magasin.
<code>fa-phone</code>	Icône représentant le numéro de téléphone.
<code>fa-envelope</code>	Icône représentant l'adresse email.
<code>fab fa-whatsapp</code>	Icône officielle WhatsApp.
<code>wa.me</code>	Créer un lien direct vers WhatsApp depuis le site.
<code>contact-grid</code>	Organiser la page contact sous forme de grille CSS.
<code>form-input</code>	Classe CSS utilisée pour styliser les champs formulaire.
<code>alert-error</code>	Affichage visuel des messages d'erreur formulaire.
<code>btn-full</code>	Bouton occupant toute la largeur disponible.
<code>required</code>	Rendre un champ obligatoire avant validation formulaire.
<code>placeholder</code>	Afficher un texte exemple dans les champs formulaire.

Fonctions et Commandes — style.css

<code>:root</code>	Créer des variables globales CSS réutilisables dans tout le site.
<code>var()</code>	Récupérer automatiquement une variable CSS.
<code>box-sizing:border-box</code>	Inclure padding et border dans la taille réelle des éléments.
<code>scroll-behavior:smooth</code>	Créer un scroll fluide entre les sections du site.
<code>max-width</code>	Limiter automatiquement la largeur maximale du contenu.
<code>margin:0 auto</code>	Centrer automatiquement un bloc horizontalement.
<code>transition</code>	Créer des animations fluides entre les changements CSS.
<code>transform</code>	Modifier visuellement un élément sans changer sa position réelle.
<code>translateY()</code>	Déplacer verticalement un élément CSS.
<code>position:fixed</code>	Fixer un élément à une position écran pendant le scroll.
<code>z-index</code>	Contrôler la priorité d'affichage des éléments.

<code>backdrop-filter:blur()</code>	Ajouter un effet flou derrière un élément transparent.
<code>display:flex</code>	Activer Flexbox pour organiser automatiquement les éléments.
<code>justify-content</code>	Organiser horizontalement les éléments Flexbox.
<code>align-items</code>	Organiser verticalement les éléments Flexbox.
<code>gap</code>	Ajouter des espaces entre les éléments Flexbox/Grid.
<code>::after</code>	Créer un pseudo-élément décoratif après un élément HTML.
<code>:hover</code>	Déclencher un style lorsque la souris passe sur un élément.
<code>opacity</code>	Contrôler le niveau de visibilité d'un élément.
<code>@keyframes</code>	Créer des animations CSS personnalisées.
<code>animation</code>	Exécuter automatiquement une animation CSS.
<code>linear-gradient()</code>	Créer un dégradé de couleurs CSS.
<code>clamp()</code>	Adapter automatiquement les tailles selon la taille d'écran.
<code>display:grid</code>	Activer CSS Grid pour organiser les éléments en grille.
<code>grid-template-columns</code>	Définir automatiquement les colonnes Grid.
<code>repeat()</code>	Répéter automatiquement plusieurs colonnes Grid.
<code>minmax()</code>	Définir une taille minimum et maximum responsive.
<code>auto-fill</code>	Ajouter automatiquement des colonnes selon l'espace disponible.
<code>overflow:hidden</code>	Masquer automatiquement le contenu dépassant.
<code>box-shadow</code>	Ajouter une ombre autour des éléments.
<code>object-fit:cover</code>	Adapter automatiquement les images sans déformation.
<code>border-radius</code>	Arrondir automatiquement les coins des éléments.
<code>@media</code>	Créer le Responsive Design pour différentes tailles d'écran.
<code>max-width:768px</code>	Appliquer des styles pour tablettes et téléphones.
<code>text-transform:uppercase</code>	Transformer automatiquement le texte en majuscules.
<code>letter-spacing</code>	Ajouter des espaces entre les lettres.
<code>line-height</code>	Contrôler l'espacement vertical entre les lignes de texte.

Fonctions et Commandes — main.js

<code>addEventListener()</code>	Détecter les actions utilisateur comme clic ou scroll.
<code>DOMContentLoaded</code>	Attendre le chargement complet du HTML avant d'exécuter JS.
<code>getElementById()</code>	Récupérer un élément HTML grâce à son ID.
<code>querySelectorAll()</code>	Récupérer plusieurs éléments HTML en même temps.
<code>classList.toggle()</code>	Ajouter ou supprimer automatiquement une classe CSS.
<code>classList.add()</code>	Ajouter une classe CSS à un élément.
<code>classList.remove()</code>	Supprimer une classe CSS d'un élément.
<code>window.scrollTo</code>	Récupérer la position du scroll de la page.
<code>forEach()</code>	Parcourir plusieurs éléments automatiquement.
<code>setInterval()</code>	Répéter automatiquement une fonction après un délai (slider).
<code>clearInterval()</code>	Arrêter un timer JavaScript en cours.
<code>setTimeout()</code>	Exécuter une fonction après un délai spécifique (flash messages).
<code>preventDefault()</code>	Bloquer le comportement par défaut d'un élément HTML.
<code>getAttribute()</code>	Récupérer la valeur d'un attribut HTML (data-filter, data-tab).
<code>style.display</code>	Modifier l'affichage d'un élément HTML (block/none).
<code>style.opacity</code>	Modifier la transparence d'un élément pour les animations.
<code>contains()</code>	Vérifier si un élément appartient à un autre élément DOM.
<code>remove()</code>	Supprimer un élément HTML du DOM après animation.
<code>openModal()</code>	Ouvrir une fenêtre popup en ajoutant la classe 'open'.
<code>closeModal()</code>	Fermer une fenêtre popup en supprimant la classe 'open'.
<code>key === 'Escape'</code>	Fermer toutes les modals ouvertes avec la touche Échap.
<code>goTo(n)</code>	Naviguer vers un slide spécifique du slider hero.
<code>startAuto()</code>	Démarrer le slider automatique toutes les 5 secondes.
<code>resetAuto()</code>	Réinitialiser le timer slider après interaction manuelle.
<code>switchTab()</code>	Changer dynamiquement entre les formulaires Login et Register.